Online Supplementary Material

EdgeVA Benchmarking Methodology

For: "Edge-Intelligent Video Analytics: A Systematic Survey of Architectures, Applications, and Deployment Strategies Across Smart City Verticals"

ACM Computing Surveys

Mukhiddin Toshpulatov [1,2] Wookey Lee [3] Jinsoo Cho [1]

[1]Gachon University, Seongnam, South Korea [2]Jizzakh Branch of NUUz, Uzbekistan
[3]Inha University, Incheon, South Korea

Corresponding: trinity@inha.ac.kr, jscho@gachon.ac.kr

Artefacts: https://github.com/muxiddin19/EdgeVA

Contents

1 Hardware Configuration **1**
1.1 Server Platform (CPU and GPU Runs) 1
1.2 Jetson Orin NX Figures (from Literature) 1

2 Model Export Procedure **1**
2.1 Model Checksums (SHA-256, truncated) 2

3 Measurement Protocol **2**

4 Pipeline-Timing Breakdown **2**

5 Confidence Intervals **3**

6 Reproducibility Checklist **3**

Purpose. This document provides the complete measurement protocol for all latency, throughput, and pipeline-timing figures in the main article (Table 2, Figure 5, and Section 6.4). All scripts, model weights, environment specifications, and raw result CSVs are available at the repository above.

1 Hardware Configuration

1.1 Server Platform (CPU and GPU Runs)

- **CPU:** Intel Core i9-14900K (24-core/32-thread, 3.2 GHz base / 6.0 GHz boost)
- **GPU:** Dual NVIDIA GeForce RTX 4090 (24 GB GDDR6X each); benchmarks pinned to GPU 0 (`device_id=0`)
- **RAM:** 125 GB DDR5
- **OS:** Ubuntu 22.04.4 LTS (kernel 6.8.0)
- **Driver / CUDA:** NVIDIA driver 535.288.01; CUDA 12.2
- **Conda environment:** isolated `edgeva` env (Python 3.10.20); key packages: `onnxruntime-gpu` 1.24.4, `ultralytics` 8.4.36, `onnxslim` 0.1.90, `onnx` 1.16.0, `scipy` 1.13.0, `numpy` 1.26.4

1.2 Jetson Orin NX Figures (from Literature)

Jetson Orin NX 16 GB results are sourced from Alqahtani et al. (2024): JetPack 5.1.2, TensorRT 8.6.2, ONNX 1.14, INT8 calibration on 500 representative images from COCO 2017 val.

2 Model Export Procedure

ONNX models were exported with the `ultralytics` Python API:

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")          # or yolov8s.pt, yolov8m.pt
model.export(format="onnx",
opset=20,
simplify=True,
dynamic=False,
imgsz=640)
```

Post-processed with `onnxslim` 0.1.90 for graph simplification. No TensorRT engine conversion was performed for server-side runs; the ONNX Runtime `CUDAExecutionProvider` with `ORT_ENABLE_ALL` graph optimisation was the only accelerator.

2.1 Model Checksums (SHA-256, truncated)

Model file	SHA-256 (first 8 chars)	Size
`yolov8n.onnx`	`b3a1f2e4`	12.3 MB
`yolov8s.onnx`	`d7f2c8a1`	42.8 MB
`yolov8m.onnx`	`e9c4b3f7`	99.0 MB

Full SHA-256 hashes and download links are in `experiments/results/gpu_summary.json` in the repository.

3 Measurement Protocol

- **Input:** random FP32 tensor $1 \times 3 \times 640 \times 640$, values $\sim \mathcal{U}(0, 1)$; batch size = 1.
- **Warmup:** 100 inference calls discarded before timing.
- **Timed runs:** 500 calls; wall-clock measured with `time.perf_counter()` bracketing each `session.run()`.
- **Statistics:** p50, p95, p99 latency (ms); FPS = 1000/p50; mean and std also recorded.
- **Threading:** `intra_op_num_threads=8` for CPU provider; GPU provider uses the CUDA default stream.
- **Thermal steady-state:** 10-minute system idle before each run; GPU junction temperature remained below 83 °C throughout.
- **Power:** GPU power sampled via `nvidia-smi --query-gpu=power.draw --format=csv` at 1 s intervals during the 500-run window. GPU 0 drew ≈68 W (YOLOv8n CUDA) to ≈112 W (YOLOv8m CUDA) during inference.
- **CPU isolation:** CPU runs used `CPUExecutionProvider` only; GPU was idle.

4 Pipeline-Timing Breakdown

The same 500-run / 100-warmup protocol was applied to a sequential five-stage pipeline:

1. **Preprocess:** BGR→RGB, HWC→BCHW, divide by 255, cast to FP32 (`numpy`).
2. **Inference:** ONNX Runtime `session.run()`.
3. **Postprocess:** confidence threshold filter (> 0.25), box decoding.
4. **Tracking:** LITE-proxy association (Hungarian algorithm, IoU cost matrix, $\tau_{\text{IoU}} = 0.45$).
5. **Analytics:** zone occupancy count (point-in-polygon test).

Each stage is timed separately; total pipeline latency = sum of all five stages (Table 2 in the main article).

5 Confidence Intervals

With 500 runs and empirical latency standard deviations of 2–5 ms (CUDA) and 3–12 ms (CPU), the 95% bootstrap confidence interval on the p50 estimate is ±0.3–0.7 ms. Bootstrap formula:

$$\text{CI}_{95} = [\hat{\theta}^*_{0.025},\ \hat{\theta}^*_{0.975}], \tag{1}$$

where $\hat{\theta}^*_\alpha$ is the α-quantile of the bootstrap distribution of the median over $B = 10{,}000$ resamples. Full distributions are in `experiments/results/gpu_summary.json`.

6 Reproducibility Checklist

Artefact	**Available in repo**
Benchmark script (`run_benchmarks_gpu.py`)	✓
Model export script (`export_models.py`)	✓
Environment spec (`setup_env.sh`)	✓
Raw CSV results (`experiments/results/`)	✓
Bootstrap summary (`gpu_summary.json`)	✓

Repository: https://github.com/muxiddin19/EdgeVA